From Phrase Structure to Type Algebra

Each Word Is Its Own Grammar Rule

Traditional generative grammar computes sentence structure using top-down phrase-structure rules that recursively combine constituents. Categorial grammar perfectly inverts this perspective: rather than specifying abstract construction rules, it assigns each lexical item a dedicated *type* that rigorously encodes its combinatory potential. For example, a transitive verb like "chases" is not loosely defined as "a word requiring a subject and object," but specifically assigned the algebraic type $(n\backslash s)/n$—an active function that categorically demands a noun to its right (the object) and a noun to its left (the subject) to yield a complete sentence.

Lambek's Residuation Law: Consuming and Producing Types in Linear Order

Lambek [-@lambek1958mathematics] laid the algebraic foundations by introducing a *residuated* structure on syntactic types. He defined two binary operations—the left residual $\backslash$ and the right residual $/$—derived from the multiplicative connective $\otimes$ (concatenation). The fundamental axiom governing this system is the residuation law:

$$A \otimes B \leq C \quad \iff \quad A \leq C/B \quad \iff \quad B \leq A\backslash C \tag{1}$$

This law brilliantly captures the fundamental duality of syntax: a verb of type $(n\backslash s)/n$ actively *consumes* available noun arguments to *produce* a well-formed sentence. The resulting **Lambek calculus** operates as a non-commutative intuitionistic linear logic—non-commutative because strict word order dictates meaning, and linear because the derivation logically consumes each lexical resource exactly once.

Pregroups Unlock Compact Closure: The Bridge to DisCoCat

Cups, Caps, and Joyal–Street: How Wires Prove Grammaticality

The pivotal connection between categorial grammar and visualization comes from Joyal and Street [-@joyalstreet1991geometry], who proved that morphisms in monoidal categories can be faithfully represented as *string diagrams*—planar graphs where:

- **Wires** represent types (objects of the category)
- **Boxes** represent words or operations (morphisms)
- **Vertical composition** represents sequential application
- **Horizontal juxtaposition** represents the tensor product (concatenation)
- **Cups and caps** represent the contraction/expansion of a pregroup

2 surveys three syntactic structures side by side (progression from intransitive to passive); 1 shows the same transitive sentence rendered by our native matplotlib pipeline—a direct computational output confirming word-box layout, wire routing, and